# FIT5149 · Code-Reading Mock Exam · ANSWER KEY

Semester 2 · Weeks 1–12 · every question gives code and asks for its output / result

**Instructions**

- 100 marks: **Section A** — 16 select-all multiple-choice questions (2 marks each = 32); **Section B** — 13 short-answer questions (68 marks).
- No computer is allowed: every answer can be derived by hand or with a scientific calculator. Randomness is always supplied explicitly (no unseeded random calls).
- Library behaviour is as in the unit notebooks (outputs were produced with Python 3.13, NumPy 2.5, pandas 3.0, scikit-learn 1.9, SciPy 1.18, statsmodels/patsy, PyTorch 2.15, pgmpy 1.1.2; `DiscreteBayesianNetwork`, `DiscreteMarkovNetwork`). Arrays are printed by NumPy defaults.
- For multiple-choice questions select **all** correct options; one or more may be correct.

| Week | Questions |
|---|---|
| Week 1 | A1 |
| Week 1–2 | A2 |
| Week 2 | A3, B1 |
| Week 3 | A4, B2 |
| Week 4 | A5, B3 |
| Week 5 | A6, A7, A8, B4 |
| Week 6 | A9, A10, B5 |
| Week 7 | A11, B6 |
| Week 8 | A12, A13, B7 |
| Week 9 | A14, B8 |
| Week 10 | A15, B9 |
| Week 11 | B10, B11, B12 |
| Week 12 | A16, B13 |

Mọi output trong tài liệu này được **chạy thật** bằng Python 3.13 (numpy 2.5, pandas 3.0, scikit-learn 1.9, pgmpy 1.1.2, PyTorch 2.15; đã kiểm tra ổn định qua nhiều PYTHONHASHSEED) rồi mới viết lời giải. Lời giải bằng tiếng Việt, thuật ngữ giữ tiếng Anh.

## Section A — Multiple choice (select all correct options) · 32 marks

**Question 1** · Week 1 · EDA & preprocessing **(2 marks)**

The code below is run. Which statements about what it prints are correct?

```
import numpy as np, pandas as pd

df = pd.DataFrame({'Area':  [50, 60, np.nan, 80, 70],
                   'Type':  ['A', 'B', 'A', np.nan, 'B'],
                   'Price': [200, 250, 230, 300, 280]})
df['Area'] = df['Area'].fillna(df['Area'].mean())
df['Type'] = df['Type'].fillna(df['Type'].mode()[0])
X = pd.get_dummies(df[['Area', 'Type']], drop_first=True)

print(df['Area'].tolist())
print(df['Type'].tolist())
print(X.shape, list(X.columns))
print(df.isna().sum().sum())
print(round(df['Area'].std(), 2), round(np.std(df['Area'].to_numpy()), 2))
```

Output (đã chạy thật):

```
[50.0, 60.0, 65.0, 80.0, 70.0]
['A', 'B', 'A', 'A', 'B']
(5, 2) ['Area', 'Type_B']
0
11.18 10.0
```

**A.** The first printed line is [50.0, 60.0, 65.0, 80.0, 70.0].
✔ **ĐÚNG** — Area có 4 giá trị hợp lệ (50, 60, 80, 70) nên mean = 260/4 = **65**; ô NaN được thay bằng 65.0 (float vì cột có NaN).

**B.** The missing Type is imputed with 'B', because B appears last in the data.
✘ **SAI** — A và B xuất hiện 2 lần mỗi loại (hoà). `mode()` trả về **cả hai** giá trị đã sắp xếp, ['A','B'], nên `[0]` = 'A'. In ra ['A','B','A','A','B'].

**C.** X has shape (5, 3) because both categories A and B receive an indicator column.
✘ **SAI** — `drop_first=True` bỏ dummy đầu tiên (Type_A) ⇒ chỉ còn Area và Type_B ⇒ shape **(5, 2)**. (Bỏ một mức để tránh dummy-variable trap/đa cộng tuyến hoàn hảo với intercept.)

**D.** The column list printed is ['Area', 'Type_B'].
✔ **ĐÚNG** — Đúng như giải thích ở trên; cột dummy mang tên `Type_B`.

**E.** The last line prints 11.18 first and 10.0 second.
✔ **ĐÚNG** — Dữ liệu sau impute: 50, 60, 65, 80, 70, tổng bình phương lệch = 500. pandas `.std()` dùng **ddof=1**: √(500/4)=11.18; numpy `np.std` dùng **ddof=0**: √(500/5)=10.0.

**Đáp án: A, D, E**

**Question 2** · Week 1–2 · Model evaluation (cross-validation) **(2 marks)**

`neg_mse` holds the scores returned by `cross_val_score(..., scoring="neg_mean_squared_error")` for three folds. Which statements about the printed output are correct?

```
import numpy as np

neg_mse = np.array([-9.0, -16.0, -25.0])      # what cross_val_score(...,
scoring="neg_mean_squared_error") returned
rmse = np.sqrt(-neg_mse)

print(rmse)
print(rmse.mean())
print(np.sqrt((-neg_mse).mean()))
print(rmse.std())
print(rmse.std(ddof=1))
```

Output (đã chạy thật):

```
[3. 4. 5.]
4.0
4.08248290463863
0.816496580927726
1.0
```

**A.** The mean of the per-fold RMSEs (4.0) is smaller than the RMSE computed from the mean MSE (4.0825).
✔ **ĐÚNG** — Per-fold RMSE = 3, 4, 5 ⇒ mean = 4.0. Mean MSE = (9+16+25)/3 = 16.67 ⇒ √ = 4.0825. Vì √ là hàm lõm (Jensen) nên mean(√MSE) ≤ √mean(MSE); hai cách **không** bằng nhau.

**B.** Averaging the per-fold RMSEs always gives the same number as taking the square root of the averaged MSE.
✘ **SAI** — Chỉ bằng nhau khi mọi fold có MSE như nhau. Ở đây 4.0 ≠ 4.0825.

**C.** `rmse.std()` prints 0.8165 because NumPy's default is ddof=0 (divide by n).
✔ **ĐÚNG** — Độ lệch bình phương quanh mean 4: 1+0+1 = 2; 2/3 = 0.667 ⇒ √ = 0.8165.

**D.** `rmse.std(ddof=1)` prints 1.0, the sample standard deviation (divide by n−1).
✔ **ĐÚNG** — 2/(3−1) = 1 ⇒ √1 = 1.0.

**E.** The prefix `neg_` means that the model's prediction errors are negative.
✘ **SAI** — scikit-learn quy ước 'score càng lớn càng tốt'; metric là loss (MSE) nên bị **đổi dấu**. Vì vậy code của môn luôn viết `-cross_val_score(...)` để lấy lại giá trị dương.

**Đáp án: A, C, D**

**Question 3** · Week 2 · Classification metrics **(2 marks)**

A logistic-regression model produced the probabilities in `prob`. Which statements are correct?

```
import numpy as np
from sklearn.metrics import confusion_matrix

y_true = np.array([0, 0, 0, 0, 0, 0, 1, 1, 1, 1])
prob   = np.array([0.10, 0.20, 0.30, 0.40, 0.55, 0.60, 0.35, 0.50, 0.80, 0.90])   # P(default = 1)
pred = (prob > 0.5).astype(int)
cm = confusion_matrix(y_true, pred)

print(pred)
print(cm)
print((pred != y_true).mean())
print(cm[1, 1] / cm[1].sum())
print(cm[1, 1] / cm[:, 1].sum())
pred2 = (prob >= 0.5).astype(int)
print(confusion_matrix(y_true, pred2)[1, 1] / 4)
```

Output (đã chạy thật):

```
[0 0 0 0 1 1 0 0 1 1]
[[4 2]
 [2 2]]
0.4
0.5
0.5
0.75
```

**A.** `cm[0, 1] = 2` counts observations whose true class is 0 but which were predicted as 1.
✔ **ĐÚNG** — `confusion_matrix(y_true, pred)`: hàng = **true**, cột = **predicted**. cm[0,1] là true=0, pred=1 (false positive): các điểm 0.55 và 0.60.

**B.** The rows of the confusion matrix correspond to the predicted classes.
✘ **SAI** — Hàng là nhãn thật, cột là nhãn dự đoán: $C_{ij}$ = số quan sát thuộc lớp i được dự đoán vào lớp j (quy ước của `sklearn.metrics.confusion_matrix`).

**C.** The printed error rate is 0.4.
✔ **ĐÚNG** — Sai = 2 (FP) + 2 (FN) = 4 trong 10 ⇒ 0.4 (cũng chính là cm[0,1]+cm[1,0] chia 10).

**D.** The observation with probability exactly 0.50 is classified as class 1 by the first rule.
✘ **SAI** — Luật là `prob > 0.5` (strict) ⇒ 0.50 bị xếp vào class 0. Đó là lý do pred = [0 0 0 0 1 1 0 0 1 1].

**E.** Replacing `>` by `>=` raises the recall of class 1 to 0.75.
✔ **ĐÚNG** — Điểm 0.50 (true = 1) chuyển thành dự đoán 1 ⇒ TP từ 2 lên 3 ⇒ recall = 3/4 = 0.75 (dòng in cuối).

**Đáp án: A, C, E**

**Question 4** · Week 3 · Non-linear methods (polynomials & splines) **(2 marks)**

Which statements about the printed output are correct?

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from patsy import dmatrix

P = PolynomialFeatures(degree=2)
print(P.fit_transform([[2, 3]]))
print(P.get_feature_names_out(['a', 'b']))
print(PolynomialFeatures(degree=2, include_bias=False, interaction_only=True).fit_transform([[2, 3]]))

x = np.linspace(0, 10, 11)
print(dmatrix("bs(x, df=6, degree=3)", {"x": x}).shape)
print(dmatrix("bs(x, knots=(3, 6), degree=3)", {"x": x}).shape)
```

Output (đã chạy thật):

```
[[1. 2. 3. 4. 6. 9.]]
['1' 'a' 'b' 'a^2' 'a b' 'b^2']
[[2. 3. 6.]]
(11, 7)
(11, 6)
```

**A.** The first printed array is [[1. 2. 3. 4. 6. 9.]].
✔ **ĐÚNG** — Với [a,b]=[2,3], degree 2 (có bias): 1, a, b, $a^2$, ab, $b^2$ = 1, 2, 3, 4, 6, 9.

**B.** With `interaction_only=True, include_bias=False` the output is [[2. 3. 4. 9.]].
✘ **SAI** — `interaction_only` bỏ các luỹ thừa ($a^2$, $b^2$) và giữ tích chéo: [a, b, ab] = [[2. 3. 6.]].

**C.** `bs(x, df=6, degree=3)` gives 7 columns because `dmatrix` adds an intercept column to the 6 basis functions.
✔ **ĐÚNG** — `df` = số hàm cơ sở (không tính intercept) = 6; patsy thêm cột hằng ⇒ (11, 7).

**D.** `bs(x, knots=(3, 6), degree=3)` gives 6 columns: an intercept plus degree + #knots = 5 basis functions.
✔ **ĐÚNG** — df = degree + số knots (khi không include_intercept) = 3 + 2 = 5, cộng intercept = 6 ⇒ (11, 6).

**E.** A cubic regression spline with two knots needs 12 free coefficients, so the design matrix would have 12 columns.
✘ **SAI** — 3 đoạn cubic × 4 hệ số = 12 trước ràng buộc; 3 ràng buộc liên tục (hàm, đạo hàm 1, đạo hàm 2) × 2 knots = 6 ràng buộc ⇒ 12 − 6 = **6** bậc tự do.

**Đáp án: A, C, D**

**Question 5** · Week 4 · Support vector machines **(2 marks)**

The code was run and printed the three lines shown. Which statements are supported by the code and its output?

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.svm import SVC

X, y = make_blobs(n_samples=200, centers=[[0, 0], [3, 3]], cluster_std=1.5, random_state=0)
for C in [0.001, 0.1, 100]:
    m = SVC(kernel='linear', C=C).fit(X, y)
    print(C, m.n_support_, round(m.score(X, y), 3))
```

Printed output:

```
0.001 [88 88] 0.92
0.1 [29 28] 0.915
100 [24 24] 0.92
```

**A.** For C = 0.001 the classifier uses 176 support vectors in total.
✔ **ĐÚNG** — `n_support_` = số support vector **theo từng class** = [88, 88] ⇒ 176 trên tổng 200 điểm (gần như mọi điểm nằm trong/ trên margin).

**B.** Increasing C from 0.001 to 100 reduces the number of support vectors, while the training accuracy barely changes.
✔ **ĐÚNG** — 88+88 → 29+28 → 24+24; accuracy 0.92 / 0.915 / 0.92.

**C.** With C = 100 no training point violates the margin, because the two classes are separable.
✘ **SAI** — Accuracy = 0.92 < 1 ⇒ dữ liệu **không** tách tuyến tính (hai blob chồng lên nhau, std = 1.5). Vẫn còn điểm vi phạm margin/bị phân sai dù C lớn.

**D.** C penalises margin violations: a small C tolerates many violations, producing a wider margin.
✔ **ĐÚNG** — C là hệ số phạt slack; C nhỏ ⇒ chấp nhận nhiều vi phạm ⇒ margin rộng ⇒ nhiều support vector.

**E.** `m.score(X, y)` reports the test accuracy.
✘ **SAI** — Nó được gọi trên chính (X, y) dùng để fit ⇒ là **training** accuracy.

**Đáp án: A, B, D**

**Question 6** · Week 5 · Neural networks (CNN shapes) **(2 marks)**

Which statements are correct?

```
import torch, torch.nn as nn

x = torch.randn(8, 3, 64, 64)                    # (batch, channels, height, width)
conv = nn.Conv2d(3, 16, kernel_size=5, stride=2, padding=2)
pool = nn.MaxPool2d(kernel_size=2, stride=2)
gap  = nn.AdaptiveAvgPool2d(1)

h = conv(x)
p = pool(h)
print(h.shape)
print(p.shape)
print(gap(p).shape)
print(nn.Flatten()(p).shape)
print(sum(q.numel() for q in conv.parameters()))
```

Output (đã chạy thật):

```
torch.Size([8, 16, 32, 32])
torch.Size([8, 16, 16, 16])
torch.Size([8, 16, 1, 1])
torch.Size([8, 4096])
1216
```

**A.** `h.shape` is (8, 16, 32, 32) because ⌊(64 + 2·2 − 5)/2⌋ + 1 = 32.
✔ **ĐÚNG** — H_out = ⌊(H_in + 2p − k)/s⌋ + 1 = ⌊63/2⌋ + 1 = 31 + 1 = 32; số kênh = 16 filters.

**B.** With stride 1 instead of 2, the output of `conv` would be (8, 16, 64, 64).
✔ **ĐÚNG** — (64 + 4 − 5)/1 + 1 = 64: padding 2 với kernel 5 là 'same padding'.

**C.** The last printed value is 1200 because a `Conv2d` layer has no bias term by default.
✘ **SAI** — `bias=True` theo mặc định: 16 × (3·5·5) = 1200 weights **+ 16 biases = 1216** (giá trị in ra).

**D.** `nn.Flatten()(p)` has 4096 features, so a following `nn.Linear(4096, 10)` would add 40,970 parameters.
✔ **ĐÚNG** — 16·16·16 = 4096 (shape (8, 4096)); Linear: 4096·10 + 10 = 40 970.

**E.** `gap(p)` keeps the 16 × 16 spatial grid, so its shape is (8, 16, 16, 16).
✘ **SAI** — `AdaptiveAvgPool2d(1)` trung bình mỗi kênh về 1×1 ⇒ (8, 16, 1, 1).

**Đáp án: A, B, D**

**Question 7** · Week 5 · Neural networks (training mechanics) **(2 marks)**

Which statements are correct?

```
import torch, torch.nn as nn

w = torch.tensor([2.0], requires_grad=True)
for _ in range(2):
    loss = (3 * w) ** 2
    loss.backward()
print(w.grad)

w.grad.zero_()
loss = (3 * w) ** 2
loss.backward()
print(w.grad)

logits = torch.tensor([[2.0, 0.0]])
target = torch.tensor([0])
ce = nn.CrossEntropyLoss()
print(round(ce(logits, target).item(), 3))
print(round(ce(torch.softmax(logits, dim=1), target).item(), 3))
```

Output (đã chạy thật):

```
tensor([72.])
tensor([36.])
0.127
0.383
```

**A.** The first printed line is `tensor([72.])`, because gradients accumulate across the two `backward()` calls.
✔ **ĐÚNG** — loss = $9w^2$ ⇒ dloss/dw = $18w = 36$ tại $w = 2$. PyTorch **cộng dồn** vào `.grad`: 36 + 36 = 72. Đây là lý do cần `optimizer.zero_grad()` mỗi iteration.

**B.** After `w.grad.zero_()` a single backward pass gives 36.
✔ **ĐÚNG** — Reset về 0 rồi cộng 36 ⇒ tensor([36.]).

**C.** `loss.backward()` updates `w` to `w − lr·grad`.
✘ **SAI** — `backward()` chỉ **tính** gradient. Việc cập nhật tham số do `optimizer.step()` thực hiện.

**D.** `CrossEntropyLoss` expects raw logits; for logits [2, 0] and target 0 it prints 0.127 = $\log(1 + e^{-2})$.
✔ **ĐÚNG** — CE = $-\log \text{softmax}(\text{logits})[0] = -\log(e^2/(e^2+1)) = \log(1 + e^{-2}) = 0.127$.

**E.** Feeding `torch.softmax(logits)` into `CrossEntropyLoss` is harmless; it returns the same loss.
✘ **SAI** — CrossEntropyLoss đã gồm log-softmax; áp softmax 2 lần làm logits bị nén về [0.88, 0.12] ⇒ loss = 0.383 ≠ 0.127 (sai và gradient yếu đi).

**Đáp án: A, B, D**

**Question 8** · Week 5 · Transformers (attention) **(2 marks)**

Which statements are correct?

```
import torch, math

Q = K = V = torch.tensor([[1.0, 0.0],
                          [0.0, 1.0]])
scores = Q @ K.T / math.sqrt(Q.size(-1))
w = torch.softmax(scores, dim=-1)
print(w.round(decimals=3))
print((w @ V).round(decimals=3))

mask = torch.tensor([[1, 0],
                     [1, 1]])                      # 1 = may attend, 0 = blocked
w2 = torch.softmax(scores.masked_fill(mask == 0, float('-inf')), dim=-1)
print(w2.round(decimals=3))
print(w.sum(dim=-1))
```

Output (đã chạy thật):

```
tensor([[0.6700, 0.3300],
        [0.3300, 0.6700]])
tensor([[0.6700, 0.3300],
        [0.3300, 0.6700]])
tensor([[1.0000, 0.0000],
        [0.3300, 0.6700]])
tensor([1., 1.])
```

**A.** Every row of the attention-weight matrix `w` sums to 1.
✔ **ĐÚNG** — softmax theo `dim=-1` chuẩn hoá từng hàng ⇒ in ra tensor([1., 1.]).

**B.** `w @ V` returns `V` unchanged.
✘ **SAI** — V = I nên `w @ V = w` = [[0.67, 0.33], [0.33, 0.67]], không phải V = I. Mỗi hàng output là trung bình có trọng số của các hàng V.

**C.** In `w2`, row 1 is [1, 0] because the blocked position receives −∞ before the softmax.
✔ **ĐÚNG** — `masked_fill(mask == 0, -inf)` ⇒ $e^{-\infty} = 0$ ⇒ toàn bộ xác suất dồn vào vị trí cho phép (kiểu causal mask).

**D.** Dividing by $\sqrt{d_k}$ makes the scores larger, which sharpens the softmax.
✘ **SAI** — Chia cho $\sqrt{d_k}$ làm scores **nhỏ đi** (1 → 0.707) để softmax không bão hoà khi $d_k$ lớn. Không scale thì hàng 1 sẽ là [0.731, 0.269] thay vì [0.67, 0.33].

**E.** Row 2 of `w2` is identical to row 2 of `w`, because position 2 may attend to both positions.
✔ **ĐÚNG** — mask hàng 2 = [1, 1] ⇒ không chặn gì ⇒ [0.33, 0.67] như cũ.

**Đáp án: A, C, E**

**Question 9** · Week 6 · K-means clustering **(2 marks)**

Which statements about the printed output are correct?

```
import numpy as np
from sklearn.cluster import KMeans

X = np.array([[1], [2], [3], [10], [11], [12]])
km = KMeans(n_clusters=2, n_init=10, random_state=0).fit(X)

print(np.sort(km.cluster_centers_.ravel()))
print(km.inertia_)
print(km.predict([[6]]) == km.labels_[0])
print(len(set(km.labels_[:3])), len(set(km.labels_[3:])))
print(KMeans(n_clusters=3, n_init=10, random_state=0).fit(X).inertia_)
```

Output (đã chạy thật):

```
[ 2. 11.]
4.0
[ True]
1 1
2.5
```

**A.** The two cluster centres are 2 and 11.
✔ **ĐÚNG** — Hai nhóm {1,2,3} và {10,11,12}; centroid = mean ⇒ 2 và 11.

**B.** `inertia_` = 4.0 is the sum of squared distances of the points to their own centre.
✔ **ĐÚNG** — $(1-2)^2+(2-2)^2+(3-2)^2 = 2$, tương tự nhóm 2 = 2 ⇒ 4.0.

**C.** The new point x = 6 is assigned to the cluster whose centre is 11.
✘ **SAI** — $|6-2| = 4 < |6-11| = 5$ ⇒ gán vào cụm tâm 2, cùng cụm với `X[0]`, vì thế dòng in ra là [ True ].

**D.** With K = 3 the inertia drops to 2.5, so inertia alone can never be used to pick K.
✔ **ĐÚNG** — K = 3: {1,2,3} → 2, {10,11} → 0.5, {12} → 0 ⇒ 2.5. Inertia luôn giảm khi K tăng (K = n ⇒ 0) nên phải dùng elbow/silhouette, không chọn K nhỏ nhất theo inertia.

**E.** Using `n_init=10` guarantees that the globally optimal partition is found.
✘ **SAI** — K-means chỉ hội tụ tới **local optimum**; n_init chạy nhiều lần khởi tạo và giữ nghiệm tốt nhất, giảm rủi ro nhưng không đảm bảo.

**Đáp án: A, B, D**

**Question 10** · Week 6 · Hierarchical clustering (scipy) **(2 marks)**

Which statements about the printed output are correct?

```
import numpy as np
from scipy.spatial.distance import pdist, squareform
from scipy.cluster.hierarchy import linkage, fcluster

X = np.array([[0.], [1.], [3.], [7.], [12.]])
d = pdist(X)
Z = linkage(d, method='single')
print(len(d))
print(Z)
print(fcluster(Z, t=2, criterion='maxclust'))
Zc = linkage(X, method='complete')
print(Zc[:, 2])
print(fcluster(Zc, t=2, criterion='maxclust'))
```

Output (đã chạy thật):

```
10
[[0. 1. 1. 2.]
 [2. 5. 2. 3.]
 [3. 6. 4. 4.]
 [4. 7. 5. 5.]]
[1 1 1 1 2]
[ 1.  3.  5. 12.]
[1 1 1 2 2]
```

**A.** `linkage` receives the condensed distance vector (10 = 5·4/2 entries), which is the correct input type.
✔ **ĐÚNG** — `pdist` trả về vector rút gọn độ dài n(n−1)/2 = 10; đây là định dạng dissimilarity mà `linkage` chấp nhận (hoặc truyền thẳng ma trận quan sát X).

**B.** Passing `squareform(d)` (a 5 × 5 matrix) to `linkage` would give the same `Z`.
✘ **SAI** — Ma trận vuông bị hiểu là **5 quan sát với 5 đặc trưng** (cụm hoá các hàng của ma trận khoảng cách) ⇒ kết quả khác, kèm ClusterWarning. Đây chính là bug trong bài Week 7 của môn.

**C.** In the single-linkage tree the last merge joins observation 4 (x = 12) to the other four points at height 5.
✔ **ĐÚNG** — Hàng cuối của Z: [4, 7, 5, 5] ⇒ obs 4 hợp với cụm 7 ở độ cao 5 (khoảng cách nhỏ nhất 12−7 = 5), cụm mới có 5 phần tử.

**D.** The final merge height of complete linkage is 12, the largest distance between a point of {0, 1, 3} and a point of {7, 12}.
✔ **ĐÚNG** — `Zc[:, 2] = [1, 3, 5, 12]`. Complete linkage dùng khoảng cách **lớn nhất** giữa hai cụm: 12 − 0 = 12.

**E.** Cutting both trees into two clusters gives the same partition.
✘ **SAI** — Single: [1 1 1 1 2] (điểm 12 đứng riêng, hiệu ứng chaining); complete: [1 1 1 2 2] ({7, 12} thành một cụm).

**Đáp án: A, C, D**

**Question 11** · Week 7 · Dimension reduction (PCA & t-SNE) **(2 marks)**

Which statements about the printed output are correct?

```
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import TSNE

X = np.array([[2, 0], [0, 1], [-2, 0], [0, -1]], dtype=float)

pca = PCA(n_components=2).fit(X)
print(pca.explained_variance_ratio_.round(2))
print(np.abs(pca.transform([[1, 1]])).round(2))
print(pca.components_.shape)

Z = StandardScaler().fit_transform(X)
print(PCA().fit(Z).explained_variance_ratio_.round(2))
print(PCA(n_components=0.75).fit(X).n_components_)
print(hasattr(TSNE(n_components=2), 'transform'))
```

Output (đã chạy thật):

```
[0.8 0.2]
[[1. 1.]]
(2, 2)
[0.5 0.5]
1
False
```

**A.** The first principal component explains 80% of the variance, because the variance of column 1 (8/3) is four times that of column 2 (2/3).
✔ **ĐÚNG** — Cột 1 chứa ±2 (tổng bình phương 8, chia n−1 = 3), cột 2 chứa ±1 (tổng 2). Hai cột không tương quan nên các PC trùng trục toạ độ ⇒ tỉ lệ 0.8 / 0.2.

**B.** After `StandardScaler` the two components explain 50% each.
✔ **ĐÚNG** — Sau chuẩn hoá hai cột có cùng phương sai, vẫn không tương quan ⇒ [0.5, 0.5]. PCA **phụ thuộc scale**, vì vậy phải scale khi các biến khác đơn vị.

**C.** `PCA(n_components=0.75)` keeps one component.
✔ **ĐÚNG** — Với số thực trong (0,1), sklearn giữ số PC nhỏ nhất sao cho cumulative explained variance ≥ 0.75; PC1 đã có 0.8 ⇒ 1.

**D.** The fitted `TSNE` object can project new observations through `transform`.
✘ **SAI** — In ra False: t-SNE chỉ có `fit_transform` (không có phép chiếu cố định cho điểm mới); PCA mới có `transform`.

**E.** `pca.components_` has one row per principal component and one column per original feature.
✔ **ĐÚNG** — Shape (n_components, n_features); ở đây (2, 2) nên cả hai chiều trùng nhau, nhưng quy ước là hàng = PC (loading vector).

**Đáp án: A, B, C, E**

**Question 12** · Week 8 · Bayesian networks (d-separation) **(2 marks)**

The network is the Student network: D → G ← I, I → S, G → L. Which statements are correct?

```
from pgmpy.models import DiscreteBayesianNetwork

m = DiscreteBayesianNetwork([('D', 'G'), ('I', 'G'), ('G', 'L'), ('I', 'S')])
print(m.is_dconnected('D', 'I'))
print(m.is_dconnected('D', 'I', observed=['L']))
print(m.is_dconnected('D', 'S'))
print(m.is_dconnected('D', 'S', observed=['G']))
print(m.is_dconnected('L', 'S', observed=['G']))
print(sorted(m.get_markov_blanket('G')))
print(sorted(m.get_markov_blanket('I')))
```

Output (đã chạy thật):

```
False
True
False
True
False
['D', 'I', 'L']
['D', 'G', 'S']
```

**A.** `D` and `I` are d-connected when nothing is observed (first printed value).
✘ **SAI** — In ra False: đường D → G ← I có G là **collider** chưa quan sát ⇒ bị chặn ⇒ D ⊥ I.

**B.** Observing `L` makes `D` and `I` d-connected again.
✔ **ĐÚNG** — L là hậu duệ của collider G; quan sát hậu duệ của collider cũng 'mở' đường (explaining away) ⇒ True.

**C.** `D` and `S` are d-connected given `G`.
✔ **ĐÚNG** — Đường D → G ← I → S: G là collider và được quan sát ⇒ mở; I là non-collider chưa quan sát ⇒ không chặn ⇒ True.

**D.** `L` and `S` are d-connected given `G`.
✘ **SAI** — Đường duy nhất L ← G ← I → S đi qua G (chuỗi, không phải collider) mà G đã quan sát ⇒ bị chặn ⇒ False.

**E.** The Markov blanket of `G` is {D, I, L}.
✔ **ĐÚNG** — Markov blanket = cha (D, I) + con (L) + cha khác của các con (L không có cha nào khác) ⇒ ['D','I','L'].

**Đáp án: B, C, E**

**Question 13** · Week 8 · Bayesian networks (CPDs & factorisation) **(2 marks)**

Which statements are correct?

```
import numpy as np
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD

m = DiscreteBayesianNetwork([('D', 'G'), ('I', 'G'), ('G', 'L'), ('I', 'S')])
m.add_cpds(
    TabularCPD('D', 2, [[0.6], [0.4]], state_names={'D': ['easy', 'hard']}),
    TabularCPD('I', 2, [[0.7], [0.3]], state_names={'I': ['low', 'high']}),
    TabularCPD('G', 3, [[0.30, 0.05, 0.90, 0.50],
                        [0.40, 0.25, 0.08, 0.30],
                        [0.30, 0.70, 0.02, 0.20]],
               evidence=['I', 'D'], evidence_card=[2, 2],
               state_names={'G': ['A', 'B', 'C'], 'I': ['low', 'high'], 'D': ['easy', 'hard']}),
    TabularCPD('S', 2, [[0.95, 0.20], [0.05, 0.80]], evidence=['I'], evidence_card=[2],
               state_names={'S': ['weak', 'strong'], 'I': ['low', 'high']}),
    TabularCPD('L', 2, [[0.10, 0.40, 0.99], [0.90, 0.60, 0.01]], evidence=['G'], evidence_card=[3],
               state_names={'L': ['weak', 'strong'], 'G': ['A', 'B', 'C']}),
)
print(m.check_model())
print(round(m.get_state_probability({'D': 'hard', 'I': 'low', 'G': 'B', 'S': 'weak', 'L': 'strong'}),
4))
print(sum((c.variable_card - 1) * int(np.prod(c.cardinality[1:])) for c in m.get_cpds()))
print(2 * 2 * 3 * 2 * 2 - 1)
```

Output (đã chạy thật):

```
True
0.0399
15
47
```

**A.** `check_model()` printing True means the CPDs are consistent with the graph (correct cardinalities, columns sum to 1).
✔ **ĐÚNG** — pgmpy kiểm tra mỗi CPD có đúng biến cha như cạnh trong graph, và mỗi cột tổng bằng 1; sai thì ném ValueError.

**B.** 0.0399 equals P(D=hard)·P(I=low)·P(G=B | I=low, D=hard)·P(S=weak | I=low)·P(L=strong | G=B).
✔ **ĐÚNG** — 0.4 × 0.7 × 0.25 × 0.95 × 0.6 = 0.0399 (chain rule theo cấu trúc mạng). Cột thứ 2 của bảng G là (I=low, D=hard) vì `evidence=['I','D']` và biến cuối (D) đổi nhanh nhất ⇒ [0.05, 0.25, 0.70].

**C.** The network has 15 independent parameters, against 47 for an unrestricted joint table.
✔ **ĐÚNG** — D: 1; I: 1; G: (3−1)·4 = 8; S: (2−1)·2 = 2; L: (2−1)·3 = 3 ⇒ 15. Bảng joint 2·2·3·2·2 = 48 ô ⇒ 47 tham số tự do. Đây là lợi ích của factorisation.

**D.** The CPD of G contributes 12 free parameters (3 states × 4 parent configurations).
✘ **SAI** — 12 là số **ô** của bảng; do mỗi cột phải có tổng 1 nên chỉ có (3−1)×4 = **8** tham số tự do.

**E.** The two numbers 15 and 47 are equal as long as the CPDs are valid probabilities.
✘ **SAI** — 15 ≠ 47; chúng chỉ bằng nhau nếu đồ thị đầy đủ (không có độc lập có điều kiện).

**Đáp án: A, B, C**

**Question 14** · Week 9 · Markov networks (factors) **(2 marks)**

Which statements about the printed output are correct?

```
from pgmpy.factors.discrete import DiscreteFactor

phi1 = DiscreteFactor(['A', 'B'], [2, 2], [3, 1, 1, 3])   # values listed as (A0B0, A0B1, A1B0, A1B1)
phi2 = DiscreteFactor(['B', 'C'], [2, 2], [2, 1, 1, 2])   # values listed as (B0C0, B0C1, B1C0, B1C1)

psi = phi1 * phi2
print(sorted(psi.scope()), psi.values.shape)
tau = psi.marginalize(['B'], inplace=False)
print(sorted(tau.scope()))
print(tau.values)
print(tau.values.sum())
tau.normalize()
print(tau.values.round(3))
print(phi1.reduce([('A', 0)], inplace=False).values)
```

Output (đã chạy thật):

```
['A', 'B', 'C'] (2, 2, 2)
['A', 'C']
[[7. 5.]
 [5. 7.]]
24.0
[[0.292 0.208]
 [0.208 0.292]]
[3. 1.]
```

**A.** `psi = phi1 * phi2` has scope ['A', 'B', 'C'] and 8 entries.
✔ **ĐÚNG** — Tích factor: $\psi(a,b,c) = \varphi_1(a,b)\cdot\varphi_2(b,c)$, scope là hợp của hai scope, shape (2,2,2).

**B.** `tau` was obtained by summing out A.
✘ **SAI** — `tau.scope()` = ['A','C'] ⇒ biến bị **marginalize là B**.

**C.** 24.0 is the partition function Z of the Markov network A − B − C with these two factors.
✔ **ĐÚNG** — $Z = \Sigma_{a,b,c}\, \varphi_1\varphi_2 = 7+5+5+7 = 24$; giá trị chưa chuẩn hoá nên không phải xác suất.

**D.** After `tau.normalize()` the entry for (A=0, C=0) equals 7/12.
✘ **SAI** — Chuẩn hoá chia cho tổng 24: 7/24 = 0.292 (in ra [[0.292 0.208],[0.208 0.292]]).

**E.** The last line shows $\varphi_1(A=0, B=0)$ and $\varphi_1(A=0, B=1)$.
✔ **ĐÚNG** — `reduce([('A',0)])` cố định A = 0 và giữ lại bảng theo B: $[\varphi_1(0,0), \varphi_1(0,1)] = [3, 1]$.

**Đáp án: A, C, E**

**Question 15** · Week 10 · Exact inference (variable elimination) **(2 marks)**

The factors are the scopes of the factors of a Markov network. Which statements about the printed output are correct?

```
def run(order):
    factors = [{'A', 'B'}, {'B', 'C'}, {'C', 'D'}, {'B', 'D'}, {'D', 'E'}]
    sizes = []
    for v in order:
        involved = [f for f in factors if v in f]
        scope = set().union(*involved)          # scope of psi before the sum
        sizes.append(len(scope))
        factors = [f for f in factors if v not in f] + [scope - {v}]
    return sizes

print(run('ABCDE'))
print(run('BDACE'))
print(max(run('ABCDE')), max(run('BDACE')))
```

Output (đã chạy thật):

```
[2, 3, 2, 2, 1]
[4, 4, 3, 2, 1]
3 4
```

**A.** With order A, B, C, D, E the largest intermediate factor has 3 variables; with B, D, A, C, E it has 4.
✔ **ĐÚNG** — Order 1: ψ có scope {A,B}, {B,C,D}, {C,D}, {D,E}, {E} ⇒ [2,3,2,2,1]. Order 2: {A,B,C,D}, {A,C,D,E}, … ⇒ [4,4,3,2,1].

**B.** The cost of variable elimination grows exponentially with the size of the largest intermediate scope, so order A, B, C, D, E is cheaper.
✔ **ĐÚNG** — Bảng ψ có $k^{|\text{scope}|}$ ô (k = số trạng thái). 3 < 4 biến ⇒ thứ tự thứ nhất rẻ hơn.

**C.** The marginal probability computed by variable elimination depends on the elimination order.
✘ **SAI** — Kết quả **không đổi** (phép cộng/nhân có tính hoán vị và kết hợp); chỉ chi phí thay đổi.

**D.** Finding the elimination order with the smallest induced width is easy (polynomial time).
✘ **SAI** — Tìm thứ tự tối ưu là NP-khó ⇒ dùng heuristic (MinFill, MinNeighbors, MinWeight… như notebook Week 10).

**E.** In order A, B, C, D, E eliminating B creates a new factor over {C, D}.
✔ **ĐÚNG** — Loại B: gộp {B} (từ bước A), {B,C}, {B,D} ⇒ scope {B,C,D} (3 biến); sau khi sum-out còn τ(C,D).

**Đáp án: A, B, E**

**Question 16** · Week 12 · Structure learning (I-equivalence) **(2 marks)**

Which statements about the code and its output are correct?

```
def skel(E):
    return {frozenset(e) for e in E}

def immor(E):
    pa = {}
    for u, v in E:
        pa.setdefault(v, set()).add(u)
    s = skel(E)
    return {(frozenset((a, b)), c) for c, ps in pa.items() for a in ps for b in ps
            if a < b and frozenset((a, b)) not in s}

G1 = [('X', 'Y'), ('Y', 'Z')]
G2 = [('Z', 'Y'), ('Y', 'X')]
G3 = [('X', 'Y'), ('Z', 'Y')]
G4 = [('Y', 'X'), ('Y', 'Z')]
for G in (G1, G2, G3, G4):
    print(sorted(map(sorted, skel(G))), len(immor(G)))
same = lambda A, B: skel(A) == skel(B) and immor(A) == immor(B)
print(same(G1, G2), same(G1, G3), same(G1, G4))
```

Output (đã chạy thật):

```
[['X', 'Y'], ['Y', 'Z']] 0
[['X', 'Y'], ['Y', 'Z']] 0
[['X', 'Y'], ['Y', 'Z']] 1
[['X', 'Y'], ['Y', 'Z']] 0
True False True
```

**A.** G1 (X → Y → Z) and G2 (Z → Y → X) are I-equivalent.
✔ **ĐÚNG** — Cùng skeleton X − Y − Z và đều 0 immorality ⇒ I-equivalent (cùng $X \perp Z \mid Y$).

**B.** G3 (X → Y ← Z) is not I-equivalent to G1, because it has an immorality (v-structure) that G1 does not have.
✔ **ĐÚNG** — G3 có 1 immorality X → Y ← Z với X, Z không kề nhau ($X \perp Z$ nhưng $X \not\perp Z \mid Y$).

**C.** G1 and G4 (X ← Y → Z) are I-equivalent.
✔ **ĐÚNG** — Common cause: cùng skeleton và 0 immorality ⇒ True (dòng in cuối: True False True).

**D.** Two DAGs with the same skeleton are always I-equivalent.
✘ **SAI** — Định lý: I-equivalent ⇔ cùng skeleton **và** cùng tập immorality. G1 và G3 có cùng skeleton nhưng khác immorality.

**E.** All four graphs imply $X \perp Z \mid Y$.
✘ **SAI** — G3 thì ngược lại: conditioning vào collider Y làm X và Z **phụ thuộc**.

**Đáp án: A, B, C**

## Section B — Short answer · 68 marks

**Question 17** · Week 2 · Regularisation (Lasso vs Ridge) **(6 marks)**

The 8×5 matrix $X$ consists of five mutually orthogonal ±1 columns, so $X^TX = 8I$. The data contain no noise. State the exact output of the code, using the facts that scikit-learn minimises $(1/2n)\|y - Xw\|^2 + \alpha\|w\|_1$ for Lasso and $\|y - Xw\|^2 + \alpha\|w\|^2$ for Ridge.

```
import numpy as np
from scipy.linalg import hadamard
from sklearn.linear_model import Lasso, Ridge

H = hadamard(8).astype(float)
X = H[:, 1:6]                              # 5 columns of +-1 that are mutually orthogonal: X.T @ X =
8 * I
beta = np.array([4.0, -3.0, 2.0, 1.0, 0.5])
y = X @ beta                               # no noise

print(np.linalg.lstsq(X, y, rcond=None)[0].round(3) + 0.0)
for a in [0.5, 1.5, 3.5]:
    print('lasso', a, Lasso(alpha=a, fit_intercept=False).fit(X, y).coef_.round(3) + 0.0)
print('ridge', 8, Ridge(alpha=8, fit_intercept=False).fit(X, y).coef_.round(3) + 0.0)
```

(a) What is printed on the first line (OLS)? Why? [1]

(b) Give the three Lasso coefficient vectors printed for α = 0.5, 1.5, 3.5, showing the formula you use. [2]

(c) Give the Ridge vector for α = 8 and state how many coefficients are exactly zero in each of the four fitted models (three Lasso, one Ridge). [2]

(d) Would Ridge with a very large α set the last coefficient exactly to zero? Explain. [1]

Output (đã chạy thật):

```
[ 4.  -3.   2.   1.   0.5]
lasso 0.5 [ 3.5 -2.5  1.5  0.5  0. ]
lasso 1.5 [ 2.5 -1.5  0.5  0.   0. ]
lasso 3.5 [0.5 0.  0.  0.  0. ]
ridge 8 [ 2.   -1.5   1.    0.5   0.25]
```

**(a)** Dữ liệu không nhiễu và y = Xβ ⇒ OLS khôi phục đúng β = `[ 4. -3. 2. 1. 0.5]`. (1 điểm)

**(b)** Vì $X^TX/n = I$ (n = 8) nên bài toán Lasso **tách theo từng hệ số**: $\hat{w}_j = \text{sign}(\beta_j)\cdot\max(|\beta_j| - \alpha, 0)$ (soft-thresholding). (1 điểm công thức + 1 điểm kết quả)

| α | $\hat{w}$ | # zeros |
|---|---|---|
| 0.5 | [3.5, −2.5, 1.5, 0.5, 0] | 1 ($\|0.5\| - 0.5 = 0$) |
| 1.5 | [2.5, −1.5, 0.5, 0, 0] | 2 |
| 3.5 | [0.5, 0, 0, 0, 0] | 4 (chỉ $\|4\| > 3.5$) |

**(c)** Ridge: $\hat{w} = (X^TX + \alpha I)^{-1}X^Ty = \beta\cdot 8/(8+\alpha) = 0.5\beta$ ⇒ `[ 2. -1.5 1. 0.5 0.25]`, **0 hệ số bằng 0**. (1 điểm giá trị + 1 điểm số lượng zeros: 1, 2, 4 cho Lasso và 0 cho Ridge)

**(d)** Không. Ridge chỉ **co đều** mọi hệ số theo hệ số 8/(8+α) > 0 với mọi α hữu hạn, nên hệ số khác 0 vẫn khác 0 (chỉ tiến dần về 0 khi α → ∞). Chỉ phạt L1 (có điểm gãy tại 0) mới cho nghiệm thưa chính xác ⇒ Lasso dùng để chọn biến. (1 điểm)

**Question 18** · Week 3 · Regression splines (degree 0) **(4 marks)**

`bs(x, knots=(5, 7), degree=0)` builds a piecewise-constant basis. Predict the output of the code. No point of `df` lies exactly at a knot.

```
import numpy as np, pandas as pd
import statsmodels.formula.api as smf

df = pd.DataFrame({'x': [1, 2, 3, 4, 6, 6.5, 8, 9, 10],
                   'y': [2, 4, 3, 3, 8, 9, 5, 7, 6]})
m = smf.ols('y ~ bs(x, knots=(5, 7), degree=0)', data=df).fit()

print(m.params.round(2).tolist())
print(m.predict(pd.DataFrame({'x': [2.5, 6.2, 9.5]})).round(2).tolist())
print(m.df_model)
print(round(m.ssr, 2))
```

(a) How many columns does the design matrix have and what do they encode? [1]

(b) What is printed on the first line (the fitted coefficients)? [1]

(c) What are the second and third lines? [1]

(d) Compute the value printed on the last line. [1]

Output (đã chạy thật):

```
[3.0, 5.5, 3.0]
[3.0, 8.5, 6.0]
2.0
4.5
```

**(a)** 3 cột: intercept + 2 chỉ báo vùng. 2 knots chia trục x thành 3 vùng {x<5}, {5≤x<7}, {x≥7}; patsy bỏ chỉ báo của vùng đầu vì đã có intercept (tránh đa cộng tuyến hoàn hảo). Do đó df_model = 3 − 1 = 2. (1 điểm)

**(b)** Với hàm bậc 0, OLS cho **trung bình theo vùng**: vùng 1 (x = 1,2,3,4): y = 2,4,3,3 ⇒ 3.0; vùng 2 (6, 6.5): 8, 9 ⇒ 8.5; vùng 3 (8, 9, 10): 5, 7, 6 ⇒ 6.0. Hệ số: intercept = 3.0 (vùng cơ sở), tiếp theo là *chênh lệch* so với vùng cơ sở: 8.5 − 3 = 5.5 và 6 − 3 = 3.0 ⇒ `[3.0, 5.5, 3.0]`. (1 điểm)

**(c)** x = 2.5 ∈ vùng 1 ⇒ 3.0; x = 6.2 ∈ vùng 2 ⇒ 8.5; x = 9.5 ∈ vùng 3 ⇒ 6.0 ⇒ `[3.0, 8.5, 6.0]`. Dòng tiếp theo là `m.df_model = 2.0`. (1 điểm)

**(d)** RSS = tổng bình phương sai lệch khỏi mean từng vùng: vùng 1: $(-1)^2+1^2+0+0 = 2$; vùng 2: $0.5^2+0.5^2 = 0.5$; vùng 3: $(-1)^2+1^2+0 = 2$ ⇒ **4.5**. (1 điểm)

**Question 19** · Week 4 · Support vector machine **(5 marks)**

Seven observations (4 red, 3 blue) in the style of the ISLP Chapter 9 hard-margin exercise are used. `C = 1e6` makes the fit effectively a hard-margin classifier. Predict the output.

```
import numpy as np
from sklearn.svm import SVC

X = np.array([[3, 4], [2, 2], [4, 4], [1, 4], [2, 1], [4, 3], [4, 1]], dtype=float)
y = np.array(['red'] * 4 + ['blue'] * 3)
m = SVC(kernel='linear', C=1e6).fit(X, y)

print(m.classes_)
print(m.n_support_)
print(sorted(m.support_.tolist()))
print(m.coef_.round(2), m.intercept_.round(2))
print(m.predict([[3, 1], [1, 1]]))
print(m.decision_function([[3, 2]]).round(2))
```

(a) What does `classes_` print, and which class has a positive decision function? [1]

(b) Give `n_support_` and the sorted `support_` list, and identify the support vectors. [1]

(c) Give `coef_` and `intercept_`, write the separating hyperplane, and state the margin width. [2]

(d) Give the two predicted classes and the decision-function value of the point (3, 2). [1]

Output (đã chạy thật):

```
['blue' 'red']
[2 2]
[1, 2, 4, 5]
[[-2.  2.]] [1.]
['blue' 'red']
[-1.]
```

**(a)** `['blue' 'red']` (sắp xếp theo alphabet). `decision_function > 0` ứng với `classes_[1]` = **red**; < 0 ứng với blue. (1 điểm)

**(b)** `n_support_ = [2 2]` (2 blue, 2 red). Chỉ số (0-based) `[1, 2, 4, 5]` ⇒ các quan sát 2, 3, 5, 6 trong đề: (2,2), (4,4) màu đỏ và (2,1), (4,3) màu xanh. Các điểm (3,4), (1,4), (4,1) nằm xa margin ⇒ dịch chuyển nhẹ không đổi siêu phẳng (chỉ support vector quyết định ranh giới). (1 điểm)

**(c)** Trên support vector: w·x + b = +1 (đỏ) và −1 (xanh). Giải: w = (−2, 2), b = 1 ⇒ `[[-2. 2.]] [1.]`. Siêu phẳng $-2X_1 + 2X_2 + 1 = 0 \Leftrightarrow X_1 - X_2 - 0.5 = 0$ (đường $X_2 = X_1 - 0.5$). Margin (độ rộng toàn phần) = $2/\|w\| = 2/(2\sqrt{2}) = 0.707$. Kiểm tra: (2,2): −4+4+1 = +1 ✓; (2,1): −4+2+1 = −1 ✓; (4,3): −8+6+1 = −1 ✓. (2 điểm)

**(d)** (3,1): −6+2+1 = −3 < 0 ⇒ blue; (1,1): −2+2+1 = +1 > 0 ⇒ red ⇒ `['blue' 'red']`. (3,2): −6+4+1 = **−1.0** — nằm đúng trên ranh giới margin phía blue. (1 điểm)

**Question 20** · Week 5 · Neural network forward & backward pass **(7 marks)**

A network with 2 inputs, 3 ReLU hidden units and 2 softmax outputs is trained on one example with the cross-entropy loss. Predict every printed line (rounded as shown).

```
import numpy as np

x  = np.array([1.0, 2.0])
W1 = np.array([[0.5, -1.0], [1.0, 0.5], [-0.5, 1.0]])      # shape (3, 2)
b1 = np.array([0.0, -1.0, 0.5])
W2 = np.array([[1.0, 0.5, -1.0], [-1.0, 2.0, 1.0]])       # shape (2, 3)
b2 = np.array([0.0, 0.5])

u = W1 @ x + b1
h = np.maximum(0, u)
z = W2 @ h + b2
p = np.exp(z - z.max()); p /= p.sum()
y = np.array([1, 0])                                       # one-hot label
loss = -np.sum(y * np.log(p))

dz  = p - y
dW2 = np.outer(dz, h)
dh  = W2.T @ dz
du  = dh * (u > 0)
dW1 = np.outer(du, x)

print(u, h, z)
print(p.round(3), round(loss, 3), np.argmax(p))
print(dW2.round(3) + 0.0)
print(dW1.round(3) + 0.0)
print(W1.size + b1.size + W2.size + b2.size)
```

(a) First line: u, h, z. [2]

(b) Second line: probabilities, loss, predicted class; is the example classified correctly? [2]

(c) The two gradient matrices dW2 and dW1. Which row of dW1 is zero and why? [2]

(d) The last line (number of parameters). [1]

Output (đã chạy thật):

```
[-1.5  1.   2. ] [0. 1. 2.] [-1.5  4.5]
[0.002 0.998] 6.002 1
[[ 0.    -0.998 -1.995]
 [ 0.     0.998  1.995]]
[[0.    0.   ]
 [1.496 2.993]
 [1.995 3.99 ]]
17
```

**(a)** $u = W_1x + b_1$: $u_1 = 0.5\cdot1 - 1\cdot2 + 0 = -1.5$; $u_2 = 1\cdot1 + 0.5\cdot2 - 1 = 1.0$; $u_3 = -0.5 + 2 + 0.5 = 2.0$ ⇒ `[-1.5 1. 2.]`. $h$ = ReLU($u$) = `[0. 1. 2.]`. $z = W_2h + b_2$: $z_1 = 0 + 0.5 - 2 + 0 = -1.5$; $z_2 = 0 + 2 + 2 + 0.5 = 4.5$ ⇒ `[-1.5 4.5]`. (2 điểm)

**(b)** softmax: $p = [1/(1+e^6), e^6/(1+e^6)]$ = `[0.002 0.998]` (0.00247, 0.99753). Nhãn $y = [1, 0]$ ⇒ loss = $-\ln(0.00247) = \ln(1+e^6)$ = **6.002**. `argmax(p) = 1` trong khi nhãn thật là lớp 0 ⇒ **phân loại sai** (loss lớn). (2 điểm)

**(c)** Gradient tại logits: $dz = p - y = [-0.9975, +0.9975]$. $dW_2 = dz\, h^T$ = [[0, −0.998, −1.995], [0, 0.998, 1.995]] (cột đầu = 0 vì $h_1 = 0$). Lan truyền ngược: $dh = W_2^T dz = [-1.995, 1.496, 1.995]$; $du = dh \odot 1[u > 0] = [0, 1.496, 1.995]$ (ReLU'(−1.5) = 0). $dW_1 = du\, x^T$ = [[0, 0], [1.496, 2.993], [1.995, 3.990]]. **Hàng 1 của $dW_1$ bằng 0** vì unit 1 có $u_1 = -1.5 < 0$ ⇒ ReLU đạo hàm 0 ('dead' với mẫu này) nên các trọng số vào unit đó không nhận gradient. (2 điểm)

**(d)** $W_1$ (3×2) = 6, $b_1$ = 3, $W_2$ (2×3) = 6, $b_2$ = 2 ⇒ **17**. (1 điểm)

**Question 21** · Week 6 · Hierarchical clustering (average linkage) **(5 marks)**

Perform the computation by hand and predict the output.

```
import numpy as np
from scipy.spatial.distance import squareform
from scipy.cluster.hierarchy import linkage, fcluster

D = np.array([[0, 2, 6, 10, 9],
              [2, 0, 5,  9, 8],
              [6, 5, 0,  4, 5],
              [10, 9, 4, 0, 3],
              [9, 8, 5,  3, 0]], dtype=float)
Z = linkage(squareform(D), method='average')
print(Z.round(2))
print(fcluster(Z, t=2, criterion='maxclust'))
print(fcluster(Z, t=3.0, criterion='distance'))
```

(a) Reproduce the four rows of Z (children, height, size). Show the updated dissimilarities you use. [3]

(b) Give the partition printed by `fcluster(Z, t=2, criterion='maxclust')`. [1]

(c) Give the partition printed with `t=3.0, criterion='distance'`. [1]

Output (đã chạy thật):

```
[[0.   1.   2.   2.  ]
 [3.   4.   3.   2.  ]
 [2.   6.   4.5  3.  ]
 [5.   7.   7.83 5.  ]]
[1 1 2 2 2]
[1 1 3 2 2]
```

**(a)** Average linkage: d(A,B) = trung bình mọi cặp giữa hai cụm.

- Gần nhất: d(0,1) = 2 ⇒ hợp {0,1} (id 5), height 2, size 2. Hàng 1: `[0, 1, 2, 2]`.
- d(5,2) = (6+5)/2 = 5.5; d(5,3) = (10+9)/2 = 9.5; d(5,4) = (9+8)/2 = 8.5. Các cặp còn lại: d(2,3) = 4, d(2,4) = 5, **d(3,4) = 3** ⇒ hợp {3,4} (id 6) ở height 3. Hàng 2: `[3, 4, 3, 2]`.
- d(5,6) = (10+9+9+8)/4 = 9; d(2,6) = (4+5)/2 = **4.5**; d(5,2) = 5.5 ⇒ hợp {2} với {3,4} (id 7) ở 4.5. Hàng 3: `[2, 6, 4.5, 3]`.
- d(5,7) = (6+10+9+5+9+8)/6 = 47/6 = 7.83 ⇒ hàng 4: `[5, 7, 7.83, 5]`.

(0.5 điểm mỗi bước hợp + 1 điểm cho các khoảng cách cập nhật)

**(b)** Cắt thành 2 cụm = bỏ phép hợp cuối ⇒ {0,1} và {2,3,4}: `[1 1 2 2 2]`. (1 điểm)

**(c)** Chỉ giữ các phép hợp có height ≤ 3: (0,1)@2 và (3,4)@3 ⇒ 3 cụm {0,1}, {2}, {3,4}: `[1 1 3 2 2]` (số nhãn tuỳ ý, cần đúng cách chia). (1 điểm)

*Lưu ý:* code dùng `squareform(D)` để đưa ma trận vuông về dạng condensed; nếu đưa thẳng D vào `linkage` thì D bị coi là dữ liệu 5 chiều và kết quả sai.

**Question 22** · Week 7 · PCA from first principles **(4 marks)**

Predict the output of the code.

```
import numpy as np

X = np.array([[1, 1], [-1, -1], [2, -2], [-2, 2]], dtype=float)
Xc = X - X.mean(axis=0)
S = Xc.T @ Xc / (len(X) - 1)
vals, vecs = np.linalg.eigh(S)
order = np.argsort(vals)[::-1]
vals, vecs = vals[order], vecs[:, order]

print(vals.round(3))
print((vals / vals.sum()).round(3))
scores = Xc @ vecs[:, 0]
print(np.abs(scores).round(3))
recon = np.outer(scores, vecs[:, 0])
print(round(float(np.sum((Xc - recon) ** 2)), 3))
```

(a) The eigenvalues printed on the first line. [1]

(b) The explained-variance ratios. [1]

(c) The absolute scores on PC1. [1]

(d) The reconstruction error on the last line. How is it related to the eigenvalues? What would change if the lines that sort `vals` were deleted? [1]

Output (đã chạy thật):

```
[5.333 1.333]
[0.8 0.2]
[0.    0.    2.828 2.828]
4.0
```

Xc = X (mean 0). S = $X^TX/3$ = [[10/3, −2], [−2, 10/3]].

**(a)** Eigenvalue = 10/3 ± 2 ⇒ `[5.333 1.333]` (eigenvector (1,−1)/√2 và (1,1)/√2). (1 điểm)

**(b)** 5.333/6.667 = 0.8 và 0.2 ⇒ `[0.8 0.2]`. (1 điểm)

**(c)** Score trên PC1 = $(x_1 - x_2)/\sqrt{2}$: (1,1) → 0; (−1,−1) → 0; (2,−2) → 4/√2 = 2.828; (−2,2) → −2.828 ⇒ |scores| = `[0. 0. 2.828 2.828]`. (1 điểm)

**(d)** Sai số tái tạo 1 PC = phần phương sai của PC2 = $(n-1)\cdot\lambda_2$ = 3 × 1.333 = **4.0** (tổng bình phương score PC2: $1.414^2 \cdot 2 = 4$). Nếu bỏ phần sắp xếp, vì `np.linalg.eigh` trả eigenvalue theo thứ tự **tăng dần**, `vecs[:, 0]` sẽ là PC2 ⇒ scores = [1.414, 1.414, 0, 0] và sai số tái tạo = 3·5.333 = 16. (1 điểm)

**Question 23** · Week 8 · Bayesian network inference (pgmpy) **(6 marks)**

The network is the "cancer" network: Pollution, Smoker → Cancer → Xray, Dyspnoea. State orders: Pollution [low, high]; Smoker, Cancer and Dyspnoea [T, F]; Xray [pos, neg]. Predict the printed output.

```
import numpy as np
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

m = DiscreteBayesianNetwork([('Pollution', 'Cancer'), ('Smoker', 'Cancer'),
                             ('Cancer', 'Xray'), ('Cancer', 'Dyspnoea')])
m.add_cpds(
    TabularCPD('Pollution', 2, [[0.9], [0.1]], state_names={'Pollution': ['low', 'high']}),
    TabularCPD('Smoker', 2, [[0.3], [0.7]], state_names={'Smoker': ['T', 'F']}),
    TabularCPD('Cancer', 2, [[0.03, 0.05, 0.001, 0.02], [0.97, 0.95, 0.999, 0.98]],
               evidence=['Smoker', 'Pollution'], evidence_card=[2, 2],
               state_names={'Cancer': ['T', 'F'], 'Smoker': ['T', 'F'], 'Pollution': ['low',
'high']}),
    TabularCPD('Xray', 2, [[0.9, 0.2], [0.1, 0.8]], evidence=['Cancer'], evidence_card=[2],
               state_names={'Xray': ['pos', 'neg'], 'Cancer': ['T', 'F']}),
    TabularCPD('Dyspnoea', 2, [[0.65, 0.3], [0.35, 0.7]], evidence=['Cancer'], evidence_card=[2],
               state_names={'Dyspnoea': ['T', 'F'], 'Cancer': ['T', 'F']}),
)
ve = VariableElimination(m)
q1 = ve.query(['Cancer'], evidence={'Smoker': 'T', 'Pollution': 'high'}, show_progress=False)
q2 = ve.query(['Cancer'], evidence={'Smoker': 'T', 'Pollution': 'high', 'Xray': 'pos', 'Dyspnoea':
'T'}, show_progress=False)
q3 = ve.query(['Cancer'], evidence={'Xray': 'pos'}, show_progress=False)
q4 = ve.query(['Xray', 'Dyspnoea'], evidence={'Cancer': 'T'}, show_progress=False)
print(q1.state_names['Cancer'], np.round(q1.values, 4))
print(np.round(q2.values, 4))
print(np.round(q3.values, 4))
print(np.round(q4.values, 4))
print(m.is_dconnected('Xray', 'Dyspnoea'), m.is_dconnected('Xray', 'Dyspnoea', observed=['Cancer']))
```

(a) Line 1 (q1): P(Cancer | Smoker=T, Pollution=high). [1]

(b) Line 2 (q2): show the Bayes calculation. [2]

(c) Line 3 (q3): P(Cancer | Xray=pos). Compare with the prior P(Cancer=T) and comment. [1]

(d) Line 4 (q4): the 2×2 table, and the independence it reveals. [1]

(e) The last line (two booleans) and its meaning. [1]

Output (đã chạy thật):

```
['T', 'F'] [0.05 0.95]
[0.3391 0.6609]
[0.0503 0.9497]
[[0.585 0.315]
 [0.065 0.035]]
True False
```

**(a)** Cancer có cha là Smoker, Pollution; bảng CPD: P(Cancer=T | S=T, P=high) = **0.05** ⇒ `['T','F'] [0.05 0.95]`. (1 điểm)

**(b)** Evidence gồm tất cả các nút ngoài Cancer ⇒ theo Bayes, các nhân tử không chứa Cancer triệt tiêu: P(C=T | e) ∝ P(C=T | S=T,P=high)·P(Xray=pos | C=T)·P(Dysp=T | C=T) = 0.05 × 0.9 × 0.65 = 0.02925; P(C=F | e) ∝ 0.95 × 0.2 × 0.3 = 0.0570. Chuẩn hoá: 0.02925/0.08625 = **0.3391** ⇒ `[0.3391 0.6609]`. (2 điểm)

**(c)** Prior: P(C=T) = Σ P(S)P(P)P(C|S,P) = 0.3·0.9·0.03 + 0.3·0.1·0.05 + 0.7·0.9·0.001 + 0.7·0.1·0.02 = 0.01163. Posterior: 0.01163·0.9 / (0.01163·0.9 + 0.98837·0.2) = 0.010467/0.208141 = **0.0503** ⇒ `[0.0503 0.9497]`. X-quang dương làm xác suất ung thư tăng ~4.3 lần (0.0116 → 0.0503) nhưng vẫn thấp vì prior rất nhỏ và X-quang có false-positive 20%. (1 điểm)

**(d)** Cho Cancer = T, Xray ⊥ Dyspnoea (hai con của cùng một cha, common cause) nên bảng = tích hai CPD: hàng Xray [pos, neg] × cột Dysp [T, F]: 0.9·0.65 = 0.585; 0.9·0.35 = 0.315; 0.1·0.65 = 0.065; 0.1·0.35 = 0.035 ⇒ `[[0.585 0.315] [0.065 0.035]]`. (1 điểm)

**(e)** `True False`: Xray và Dyspnoea d-connected khi chưa quan sát gì (đường Xray ← Cancer → Dyspnoea mở), nhưng bị chặn khi Cancer được quan sát (non-collider đã quan sát) ⇒ Xray ⊥ Dyspnoea | Cancer. (1 điểm)

**Question 24** · Week 9 · Conditional random field (brute force vs variable elimination) **(6 marks)**

A three-token linear-chain CRF with labels {O, P} defines score(y) = $\prod_t$ emit[t, $y_t$] · $\prod_t$ trans[$y_{t-1}$, $y_t$]. Predict the output of the code.

```
import itertools
import numpy as np

emit  = np.array([[4, 1], [2, 3], [1, 5]], dtype=float)    # emit[t, label],  label 0 = O, 1 = P
trans = np.array([[2, 1], [1, 3]], dtype=float)            # trans[prev, cur]

def score(y):
    s = np.prod([emit[t, y[t]] for t in range(3)])
    s *= np.prod([trans[y[t - 1], y[t]] for t in range(1, 3)])
    return s

scores = {y: score(y) for y in itertools.product([0, 1], repeat=3)}
Z = sum(scores.values())
best = max(scores, key=scores.get)

print(len(scores), Z)
print(best, scores[best], round(scores[best] / Z, 4))
print(round(sum(v for y, v in scores.items() if y[1] == 1) / Z, 4))
```

(a) What is the first number on line 1, and what does Z equal? [2]

(b) Give the best label sequence, its score and its probability (line 2). [1]

(c) What is printed on line 3? Say which probability it is. [1]

(d) Show that Z can be obtained by sum-product variable elimination in the order Y3, Y2, and compare the cost with brute force. [2]

Output (đã chạy thật):

```
8 462.0
(0, 1, 1) 180.0 0.3896
0.7273
```

**(a)** $2^3$ = **8** chuỗi nhãn. Điểm từng chuỗi (O=0, P=1): (0,0,0)=32, (0,0,1)=80, (0,1,0)=12, **(0,1,1)=180**, (1,0,0)=4, (1,0,1)=10, (1,1,0)=9, (1,1,1)=135 ⇒ Z = **462** (hằng chuẩn hoá). (2 điểm)

**(b)** Điểm cao nhất là (0,1,1) = O,P,P với 180 ⇒ `(0, 1, 1) 180.0 0.3896` (= 180/462). Đây là MAP assignment (Viterbi cho cùng kết quả). (1 điểm)

**(c)** $P(Y_2 = P \mid x)$ = (12 + 180 + 9 + 135)/462 = 336/462 = **0.7273** (marginal của nhãn token giữa). (1 điểm)

**(d)** Loại $Y_3$: $\tau_3(y_2) = \Sigma_{y_3}$ trans$(y_2,y_3)\cdot$emit$_3(y_3)$ = [2·1 + 1·5, 1·1 + 3·5] = [7, 16]. Loại $Y_2$: $\tau_2(y_1) = \Sigma_{y_2}$ trans$(y_1,y_2)\cdot$emit$_2(y_2)\cdot\tau_3(y_2)$ = [2·2·7 + 1·3·16, 1·2·7 + 3·3·16] = [76, 158]. $Z = \Sigma_{y_1}$ emit$_1(y_1)\cdot\tau_2(y_1)$ = 4·76 + 1·158 = **462** ✓. Brute force: $K^T$ = 8 số hạng (tăng mũ theo độ dài câu); variable elimination trên chuỗi: $O(T\cdot K^2)$ = 3·4, tuyến tính theo T. (2 điểm)

**Question 25** · Week 10 · Variable elimination on a chain **(5 marks)**

A chain A → B → C → D has binary variables. Predict every printed line.

```
import numpy as np

pA   = np.array([0.6, 0.4])
pB_A = np.array([[0.7, 0.3], [0.2, 0.8]])      # rows: A, columns: B
pC_B = np.array([[0.9, 0.1], [0.4, 0.6]])      # rows: B, columns: C
pD_C = np.array([[0.5, 0.5], [0.1, 0.9]])      # rows: C, columns: D

tau1 = pA @ pB_A
tau2 = tau1 @ pC_B
pD   = tau2 @ pD_C
print(tau1, tau2, pD)

m = np.einsum('a,ab,bc,c->a', pA, pB_A, pC_B, pD_C[:, 1])
print(m.round(3), m.sum().round(3))
print((m / m.sum()).round(4))
```

(a) Line 1: tau1, tau2 and pD. State what each vector represents. [2]

(b) Lines 2 and 3: the vector `m`, the sum, and the posterior P(A | D=1). [2]

(c) Count the scalar multiplications needed for the three vector–matrix products versus enumerating all joint entries; what is the general complexity for a chain of n variables with k states? [1]

Output (đã chạy thật):

```
[0.5 0.5] [0.65 0.35] [0.36 0.64]
[0.36 0.28] 0.64
[0.5625 0.4375]
```

**(a)** tau1(b) = $\Sigma_a$ P(a)P(b|a) = **P(B)** = [0.6·0.7+0.4·0.2, 0.6·0.3+0.4·0.8] = [0.5, 0.5]. tau2(c) = Σ_b P(b)P(c|b) = **P(C)** = [0.5·0.9+0.5·0.4, 0.5·0.1+0.5·0.6] = [0.65, 0.35]. pD(d) = Σ_c P(c)P(d|c) = **P(D)** = [0.65·0.5+0.35·0.1, 0.65·0.5+0.35·0.9] = [0.36, 0.64]. (2 điểm)

**(b)** `m` = unnormalised P(A, D=1): đẩy vector cột D=1 = [0.5, 0.9] ngược lên: Σ_c P(c|b)·[0.5,0.9] = [0.9·0.5+0.1·0.9, 0.4·0.5+0.6·0.9] = [0.54, 0.74]; Σ_b P(b|a)·(...) = [0.7·0.54+0.3·0.74, 0.2·0.54+0.8·0.74] = [0.60, 0.70]; nhân P(a) ⇒ `m = [0.36 0.28]`. Tổng = **0.64 = P(D=1)** (khớp pD[1]) ⇒ P(A | D=1) = [0.36, 0.28]/0.64 = `[0.5625 0.4375]`. (2 điểm)

**(c)** Mỗi phép vector × ma trận 2×2: 4 phép nhân ⇒ 3 × 4 = 12. Liệt kê bảng joint: $2^4$ = 16 ô, mỗi ô 3 phép nhân = 48. Tổng quát chuỗi n biến: $O(n\cdot k^2)$ so với $O(k^n)$: tiết kiệm nhờ **distributive law** (đẩy phép tổng vào trong). (1 điểm)

**Question 26** · Week 11 · Forward sampling and rejection sampling **(5 marks)**

The uniform draws are supplied in `U`, one row per sample; each variable uses the next number of its row. `draw` returns the first state k with u ≤ cumulative probability. Predict the output.

```
import numpy as np

P_G = {(0, 0): [.30, .40, .30], (0, 1): [.05, .25, .70],    # key = (I, D); entries over G = 0, 1, 2
       (1, 0): [.90, .08, .02], (1, 1): [.50, .30, .20]}
P_S1 = {0: 0.05, 1: 0.80}                                   # P(S = 1 | I)
P_L1 = {0: 0.90, 1: 0.60, 2: 0.01}                          # P(L = 1 | G)

def draw(probs, u):                 # first state k with u <= cumulative probability
    return int(np.argmax(u <= np.cumsum(probs)))

def forward_sample(u):              # order D, I, G, S, L
    d = draw([0.6, 0.4], u[0])
    i = draw([0.7, 0.3], u[1])
    g = draw(P_G[(i, d)], u[2])
    s = draw([1 - P_S1[i], P_S1[i]], u[3])
    l = draw([1 - P_L1[g], P_L1[g]], u[4])
    return d, i, g, s, l

U = [[0.31, 0.18, 0.60, 0.92, 0.45],
     [0.75, 0.55, 0.20, 0.10, 0.95],
     [0.10, 0.90, 0.50, 0.60, 0.05],
     [0.80, 0.20, 0.40, 0.77, 0.15]]
samples = [forward_sample(u) for u in U]
kept = [s for s in samples if s[3] == 1 and s[4] == 0]      # evidence S = 1, L = 0

for s in samples: print(s)
print(kept)
print(len(kept) / len(samples))
```

(a) Give the four printed samples and show how the first one is generated. [2]

(b) What are the last two lines? [1]

(c) Compute exactly P(S = 1, L = 0). What is the expected number of accepted samples out of 1000 forward samples, and why is this approach inefficient when the evidence is rare? [2]

Output (đã chạy thật):

```
(0, 0, 1, 0, 1)
(1, 0, 1, 0, 1)
(0, 1, 0, 1, 0)
(1, 0, 2, 0, 0)
[(0, 1, 0, 1, 0)]
0.25
```

**(a)** Mẫu 1 (u = 0.31, 0.18, 0.60, 0.92, 0.45): D: cum [0.6, 1] ⇒ 0.31 ≤ 0.6 ⇒ **d = 0**; I: 0.18 ≤ 0.7 ⇒ **i = 0**; G | (I=0, D=0) = [.30, .40, .30], cum [.30, .70, 1] ⇒ 0.60 ∈ (0.30, 0.70] ⇒ **g = 1**; S | I=0: [0.95, 0.05], cum [0.95, 1] ⇒ 0.92 ≤ 0.95 ⇒ **s = 0**; L | G=1: P(L=1) = 0.6 ⇒ [0.4, 0.6], cum [0.4, 1] ⇒ 0.45 > 0.4 ⇒ **l = 1**. Kết quả 4 mẫu: `(0,0,1,0,1)`, `(1,0,1,0,1)`, `(0,1,0,1,0)`, `(1,0,2,0,0)`. (2 điểm)

**(b)** Chỉ mẫu 3 có S = 1 và L = 0 ⇒ `kept = [(0, 1, 0, 1, 0)]`, tỉ lệ chấp nhận **0.25** (1/4). (1 điểm)

**(c)** $P(s^1, l^o) = \Sigma_i P(i)P(s^1|i)\ \Sigma_g P(l^o|g)\ \Sigma_d P(d)P(g|i,d)$. i = 0: P(g) = [0.2, 0.34, 0.46] ⇒ $\Sigma_g P(l^o|g)P(g) = 0.1\cdot0.2 + 0.4\cdot0.34 + 0.99\cdot0.46 = 0.6114$. i = 1: P(g) = [0.74, 0.168, 0.092] ⇒ 0.074 + 0.0672 + 0.09108 = 0.23228. ⇒ $0.7\cdot0.05\cdot0.6114 + 0.3\cdot0.8\cdot0.23228 = 0.0214 + 0.0557 =$ **0.0771**. Kỳ vọng 1000 × 0.0771 ≈ **77** mẫu được giữ; 92% công sức bị bỏ. Evidence càng hiếm (ví dụ P(e) = 0.001 ⇒ 10 trên 10 000 mẫu) thì rejection càng lãng phí và ước lượng càng nhiễu ⇒ dùng Gibbs sampling (hoặc likelihood weighting) để *giữ cố định* evidence. (2 điểm)

**Question 27** · Week 11 · Gibbs sampling with evidence **(5 marks)**

Gibbs sampling is run in the Student network with evidence S = 1 and L = 0 held fixed. The state is (D, I, G) and each variable is resampled in the order D, I, G. Predict the output.

```
import numpy as np

PD, PI = np.array([0.6, 0.4]), np.array([0.7, 0.3])
PG  = {(0, 0): [.30, .40, .30], (0, 1): [.05, .25, .70], (1, 0): [.90, .08, .02], (1, 1): [.50, .30,
.20]}  # key (i, d)
PS1 = np.array([0.05, 0.80])            # P(S = 1 | I)
PL0 = np.array([0.10, 0.40, 0.99])      # P(L = 0 | G)

def draw(p, u): return int(np.argmax(u <= np.cumsum(p)))
def cond_D(g, i): w = np.array([PG[(i, d)][g] * PD[d] for d in (0, 1)]);                  return w /
w.sum()
def cond_I(g, d): w = np.array([PG[(i, d)][g] * PS1[i] * PI[i] for i in (0, 1)]);        return w /
w.sum()
def cond_G(d, i): w = np.array([PL0[g] * PG[(i, d)][g] for g in range(3)]);               return w /
w.sum()

d, i, g = 0, 0, 1                        # evidence S = 1, L = 0 is held fixed
for u in [(0.80, 0.50, 0.70), (0.30, 0.20, 0.95)]:
    p = cond_D(g, i); d = draw(p, u[0]); print(p.round(3), d)
    p = cond_I(g, d); i = draw(p, u[1]); print(p.round(3), i)
    p = cond_G(d, i); g = draw(p, u[2]); print(p.round(3), g)
    print((d, i, g))
```

(a) The first sweep (lines 1–4): show the first two conditional distributions you computed by hand. [2]

(b) The final state after the second sweep. [1]

(c) Why are S and L never resampled, and why does `cond_D` not involve S or L? [1]

(d) After thousands of sweeps, what would the fraction of sweeps with I = 1 estimate? Name two practical issues. [1]

Output (đã chạy thật):

```
[0.706 0.294] 1
[0.108 0.892] 1
[0.136 0.326 0.538] 2
(1, 1, 2)
[0.13 0.87] 1
[0.338 0.662] 0
[0.006 0.125 0.868] 2
(1, 0, 2)
```

**(a)** Trạng thái đầu (d,i,g) = (0,0,1). **P(D | g=1, i=0)** ∝ P(g=1 | i=0, D)·P(D) = [0.40·0.6, 0.25·0.4] = [0.24, 0.10] ⇒ [0.706, 0.294]; u = 0.80 > 0.706 ⇒ **d = 1**. **P(I | g=1, d=1, s=1)** ∝ P(g | I, d=1)·P(s=1 | I)·P(I) = [0.25·0.05·0.7, 0.30·0.80·0.3] = [0.00875, 0.072] ⇒ [0.108, 0.892]; u = 0.50 ⇒ **i = 1**. **P(G | I=0, d=1, i=1)** ∝ P(I=0 | G)·P(G | i=1, d=1) = [0.1·0.5, 0.4·0.3, 0.99·0.2] = [0.05, 0.12, 0.198] ⇒ [0.136, 0.326, 0.538]; u = 0.70 ⇒ **g = 2**. Sweep 1 kết thúc ở (1,1,2). (2 điểm)

**(b)** Sweep 2: P(D | g=2, i=1) = [0.13, 0.87], u = 0.30 ⇒ d = 1; P(I | g=2, d=1) = [0.338, 0.662], u = 0.20 ⇒ **i = 0**; P(G | d=1, i=0) = [0.006, 0.125, 0.868], u = 0.95 ⇒ g = 2. Trạng thái: `(1, 0, 2)`. (1 điểm)

**(c)** S, L là **evidence** nên được gán cố định (ta lấy mẫu P(X | E = e), X = {D, I, G}). Trong `cond_D` chỉ còn các nhân tử chứa D: P(D) và P(G | I, D) (Markov blanket của D: con G và cha khác của G là I); các nhân tử P(I), P(S|I), P(L|G) không phụ thuộc D nên **triệt tiêu khi chuẩn hoá**. (1 điểm)

**(d)** Ước lượng **P(I = 1 | S = 1, L = 0) ≈ 0.7226** (giá trị đúng tính bằng enumeration; một lần chạy 60 000 sweeps cho ≈ 0.72). Hai vấn đề thực tế: (1) **burn-in** — bỏ các mẫu đầu vì còn phụ thuộc vào khởi tạo; (2) các mẫu liên tiếp **tự tương quan** (autocorrelation ≈ 0.2 cho I) nên số mẫu hiệu dụng nhỏ hơn số sweep. (1 điểm)

**Question 28** · Week 11 · Rejection sampling (univariate) **(5 marks)**

We want samples from the density proportional to the unnormalised target $\tilde{p}(z) = 4z(1 - z)$ on [0, 1], using the Uniform(0, 1) proposal $q(z) = 1$ and the rejection-sampling recipe from the Week 11 notebook: draw $z_0 \sim q$, draw $u_0 \sim \text{Uniform}(0, k \cdot q(z_0))$, accept $z_0$ if $u_0 < \tilde{p}(z_0)$. The random numbers are supplied explicitly. Predict the output.

```
import numpy as np

p_tilde = lambda z: 4 * z * (1 - z)         # unnormalised target on [0, 1]
q = lambda z: np.ones_like(z)               # proposal Uniform(0, 1): density 1
k = 1.0                                     # chosen so that k * q(z) >= p_tilde(z) for every z

z0 = np.array([0.10, 0.50, 0.85, 0.30, 0.95, 0.65, 0.20, 0.40])   # draws from the proposal q
u  = np.array([0.50, 0.90, 0.10, 0.80, 0.30, 0.45, 0.35, 0.99])   # independent Uniform(0, 1) draws
u0 = u * k * q(z0)                                                  # u0 ~ Uniform(0, k q(z0))
accept = u0 < p_tilde(z0)

print(p_tilde(z0).round(2))
print(accept.astype(int))
print(z0[accept], accept.mean())
print(round(z0[accept].mean(), 3))
print(round((2 / 3) / k, 3), round((2 / 3) / 2.0, 3))
```

(a) Give lines 1 and 2: the values $\tilde{p}(z_0)$ and the accept/reject indicators. Show how one accepted and one rejected draw are decided. [2]

(b) Give line 3 (accepted samples and the empirical acceptance rate). [1]

(c) Give line 4, and explain why the sample mean of the accepted values is a valid estimate of E[Z] under the *normalised* target even though $\tilde{p}$ is unnormalised. [1]

(d) Line 5 prints the theoretical acceptance rates for k = 1 and k = 2. Derive them, and explain what happens to the samples if k is chosen too small (e.g. k = 0.5). [1]

Output (đã chạy thật):

```
[0.36 1.   0.51 0.84 0.19 0.91 0.64 0.96]
[0 1 1 1 0 1 1 0]
[0.5  0.85 0.3  0.65 0.2 ] 0.625
0.5
0.667 0.333
```

**(a)** $\tilde{p}(z) = 4z(1-z)$: z = 0.10 → 0.36; 0.50 → 1.00; 0.85 → 0.51; 0.30 → 0.84; 0.95 → 0.19; 0.65 → 0.91; 0.20 → 0.64; 0.40 → 0.96 ⇒ `[0.36 1. 0.51 0.84 0.19 0.91 0.64 0.96]`. Vì $k \cdot q(z) = 1$ nên $u_0 = u$. Ví dụ **z = 0.50**: $u_0 = 0.90 < 1.00$ ⇒ **accept**; ví dụ **z = 0.95**: $u_0 = 0.30 > 0.19$ ⇒ **reject** (z gần biên có $\tilde{p}$ nhỏ nên hay bị loại). Indicators: `[0 1 1 1 0 1 1 0]` (đủ điểm nếu đúng cả hai dòng và có giải thích 1 mẫu accept, 1 mẫu reject). (2 điểm)

**(b)** 5 mẫu được giữ: `[0.5 0.85 0.3 0.65 0.2 ] 0.625` (các mẫu 0.5, 0.85, 0.3, 0.65, 0.2), acceptance rate = 5/8 = **0.625**. (1 điểm)

**(c)** Trung bình = (0.5 + 0.85 + 0.3 + 0.65 + 0.2)/5 = **0.5**. Rejection sampling chỉ cần $\tilde{p}$ **đến một hằng số nhân**: xác suất chấp nhận z là $\tilde{p}(z)/(k \cdot q(z))$ nên mật độ của các mẫu được giữ tỉ lệ với $q(z) \cdot \tilde{p}(z)/(k \cdot q(z)) \propto \tilde{p}(z)$; hằng số chuẩn hoá $Z_p$ bị triệt tiêu khi chuẩn hoá ⇒ các mẫu giữ lại là mẫu từ $p(z) = \tilde{p}(z)/Z_p = 6z(1-z)$ (Beta(2,2), kỳ vọng đúng = 0.5 do đối xứng). Với 5 mẫu, trùng khớp 0.5 là may mắn; nói chung có sai số Monte Carlo ~ $1/\sqrt{n}$. (1 điểm)

**(d)** Acceptance rate lý thuyết = $\int \tilde{p}(z)dz \,/\, (k \cdot \int q(z)dz) = (2/3)/k$ ($\int_0^1 4z(1-z)dz = 2/3$) ⇒ k = 1: **0.667**; k = 2: **0.333**. Với k = 2 mẫu vẫn đúng phân phối nhưng **lãng phí** gấp đôi (k càng lớn so với mức cần thiết thì càng nhiều reject). Nếu k quá nhỏ (k = 0.5) thì $k \cdot q(z) = 0.5 < \tilde{p}(z)$ quanh $z \in (0.15, 0.85)$: xác suất chấp nhận bị cắt ở 1 ($u_0 < 0.5 < \tilde{p}$) nên vùng gần mode bị **lấy thiếu** (đỉnh bị 'phẳng' đi) ⇒ phân phối mẫu sai. Điều kiện $k \cdot q(z) \geq \tilde{p}(z)$ với mọi z là bắt buộc; trong nhiều chiều k tăng rất nhanh nên rejection sampling gần như vô dụng ⇒ cần MCMC (Gibbs). (1 điểm)

**Question 29** · Week 12 · Learning structure (PC) and parameters **(5 marks)**

Part 1 runs the skeleton phase of the PC algorithm with a p-value "oracle" (α = 0.05; every test not in `table` has p = 0.0005 and therefore rejects independence). Part 2 estimates parameters from data.

```
import itertools

V = ['X', 'Y', 'Z', 'W']
alpha = 0.05
table = {(('X', 'Y'), ()): 0.70, (('W', 'X'), ('Z',)): 0.60, (('W', 'Y'), ('Z',)): 0.50}   # every
other test: p = 0.0005

def pvalue(a, b, S):
    return table.get((tuple(sorted((a, b))), tuple(sorted(S))), 0.0005)

edges = {frozenset(e) for e in itertools.combinations(V, 2)}
sepset = {}
for size in range(3):
    for e in sorted(edges, key=lambda e: sorted(e)):
        if e not in edges: continue
        a, b = sorted(e)
        cand = sorted(v for v in V if v not in e and (frozenset((a, v)) in edges or frozenset((b, v))
in edges))
        for S in itertools.combinations(cand, size):
            if pvalue(a, b, S) > alpha:
                edges.discard(e); sepset[(a, b)] = S
                break

print(sorted(map(sorted, edges)))
print(sepset)
colliders = []
for z in V:
    nb = sorted(v for v in V if frozenset((v, z)) in edges)
    for a, b in itertools.combinations(nb, 2):
        if frozenset((a, b)) not in edges and z not in sepset[(a, b)]:
            colliders.append((a, z, b))
print(colliders)
```

Part 2:

```
import pandas as pd
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.parameter_estimator import DiscreteMLE, DiscreteBayesianEstimator

df = pd.DataFrame({'A': [0, 0, 0, 0, 1, 1, 1, 1],
                   'B': [0, 0, 0, 1, 1, 1, 1, 1]})
m1 = DiscreteBayesianNetwork([('A', 'B')])
m1.fit(df, estimator=DiscreteMLE())
print(m1.get_cpds('B').values.round(3))

m2 = DiscreteBayesianNetwork([('A', 'B')])
m2.fit(df, estimator=DiscreteBayesianEstimator(prior_type='K2'))
print(m2.get_cpds('B').values.round(3))
```

(a) Part 1: give the three printed lines. [1]

(b) Orient the skeleton: identify the collider, then apply Meek's rule to the remaining edge, and give the final directed graph. [2]

(c) Part 2: give the two printed CPD tables and explain the danger of the first one when predicting an instance with A = 1, B = 0. [2]

Output (đã chạy thật):

```
[['W', 'Z'], ['X', 'Z'], ['Y', 'Z']]
{('X', 'Y'): (), ('W', 'X'): ('Z',), ('W', 'Y'): ('Z',)}
[('X', 'Z', 'Y')]
--- part 2 ---
[[0.75 0.  ]
 [0.25 1.  ]]
[[0.667 0.167]
 [0.333 0.833]]
```

**(a)** Size 0: chỉ cặp (X,Y) có p = 0.70 > 0.05 ⇒ xoá cạnh X–Y, sepset = ø. Size 1: (W,X) cho Z: p = 0.60 ⇒ xoá, sepset {Z}; (W,Y) cho Z: p = 0.50 ⇒ xoá, sepset {Z}; các cạnh còn lại có p = 0.0005 ⇒ giữ. In ra: `[['W','Z'],`

`['X','Z'], ['Y','Z']]`, `{('X','Y'): (), ('W','X'): ('Z',), ('W','Y'): ('Z',)}`, `[('X','Z','Y')]`. (1 điểm)

**(b)** Unshielded triple X – Z – Y: X, Y không kề nhau và Z **∉ sepset(X,Y) = ø** ⇒ collider **X → Z ← Y**. Hai triple còn lại (X–Z–W, Y–Z–W) có Z ∈ sepset ⇒ không phải collider. Cạnh Z – W: X → Z và X không kề W ⇒ nếu Z ← W thì tạo ra v-structure X → Z ← W mà dữ liệu không ủng hộ (sepset(X,W) chứa Z) ⇒ **Meek rule 1** định hướng **Z → W**. Đồ thị cuối: X → Z, Y → Z, Z → W. (2 điểm)

**(c)** Đếm: A = 0: (B=0: 3, B=1: 1) ⇒ [0.75, 0.25]; A = 1: (B=0: 0, B=1: 4) ⇒ [0, 1]. MLE in `[[0.75 0.] [0.25 1.]]` (hàng = B, cột = A). Prior K2 cộng 1 pseudo-count mỗi ô: A=0 ⇒ (4,2)/6 = [0.667, 0.333]; A=1 ⇒ (1,5)/6 = [0.167, 0.833] ⇒ `[[0.667 0.167] [0.333 0.833]]`. Nguy cơ của MLE: P(B=0 | A=1) = 0 ⇒ mẫu A = 1, B = 0 có likelihood 0 (log-likelihood −∞), mô hình quá tin vào dữ liệu hiếm; smoothing (Laplace/K2/BDeu) tránh xác suất 0. (2 điểm)